Déploiement d'un modèle d'IA : comparaison de quatre pipelines d'inférence CPU et GPU

TP1 — Déploiement IA — EIGSI

ResNet-18 pré-entraîné sur ImageNet · 50 images CIFAR-10 · Google Colab

Dépôt : https://github.com/Romain-LM-Dev/TP1_deploiement_IA

Pipeline	Chaîne	Débit moyen (img/s)	Accélération vs eager
CPU n°1	PyTorch → ONNX → OpenVINO → oneDNN	17.8	×1.37
CPU n°2	PyTorch → TorchScript	14.4	×1.25
	PyTorch → torch.compile (Inductor)	15.6	×1.35
GPU n°1	PyTorch → ONNX → TensorRT FP32 → CUDA/cuDNN	1212	×1.51
	PyTorch → ONNX → TensorRT FP16	4095	×5.10
GPU n°2	PyTorch → TorchScript → CUDA	916	×1.10
	PyTorch → torch.compile (Inductor) → CUDA	539	×0.65

Synthèse : débit moyen sur 10 exécutions et accélération par rapport à PyTorch eager mesuré dans la même session. Précision Top-1 identique (64 %) pour toutes les variantes.

1. Objectif

Un même modèle peut être exécuté par des chaînes logicielles très différentes : le framework d'entraînement lui-même (PyTorch), un compilateur intégré au framework (TorchScript, torch.compile) ou un moteur d'inférence spécialisé alimenté par un format d'échange (ONNX vers OpenVINO ou TensorRT). Ce TP compare quatre de ces chaînes, deux sur CPU et deux sur GPU, sur le même modèle et les mêmes données. Il évalue le débit, le temps d'inférence, la précision, la cohérence des prédictions, l'énergie et la stabilité des mesures.

2. Les quatre pipelines

Niveau	CPU n°1	CPU n°2	GPU n°1	GPU n°2
Haut niveau	PyTorch	PyTorch	PyTorch	PyTorch
Format	ONNX (opset 17)	TorchScript ; graphe FX (TorchDynamo)	ONNX (opset 17), FP32 et FP16	TorchScript ; graphe FX (TorchDynamo)
Middle level	OpenVINO Runtime : fusions, choix des kernels	JIT freeze + optimize_for_inference ; TorchInductor	TensorRT : fusion de couches, sélection de tactiques	JIT freeze ; TorchInductor (kernels Triton)
Bas niveau	oneDNN (brgconv AVX2), MLAS	oneDNN (MKLDNN), C++/OpenMP généré	cuDNN, kernels xmma (Tensor Cores en FP16)	cuDNN, Triton
Matériel	CPU Colab	CPU Colab	GPU T4 Colab	GPU T4 Colab

CPU n°1 — OpenVINO. L'export ONNX produit 49 nœuds : les BatchNorm sont déjà repliées dans les convolutions par le repliement de constantes de l'exporteur. OpenVINO importe ce graphe (305 opérations après décomposition), puis la compilation le réduit à 112 opérations exécutées. Les 20 convolutions sont fusionnées avec leur ReLU et leur addition résiduelle, et exécutées par des kernels oneDNN `brgconv_avx2` (multiplications de matrices par blocs vectorisées AVX2). La compilation ne prend que 0.55 s.

CPU n°2 — TorchScript et torch.compile. TorchScript trace le modèle, gèle les poids et applique `optimize_for_inference` : les 20 `batch_norm` et 20 convolutions du graphe tracé disparaissent au profit de convolutions MKLDNN pré-empaquetées, avec les BN repliées. torch.compile capture le modèle en un seul graphe (aucune rupture, 69 opérations). Inductor fusionne ensuite les opérations élémentaires en boucles C++/OpenMP et délègue les convolutions à oneDNN.

GPU n°1 — TensorRT. Les 49 nœuds ONNX deviennent 25 couches dans le moteur FP32. Par exemple, `conv2 + Add + ReLU` d'un bloc résiduel forment un seul kernel. Pour chaque couche, TensorRT chronomètre plusieurs implémentations et garde la plus rapide : Winograd cuDNN en FP32, kernels `sm75_xmma_…_tensor16x8x8` sur Tensor Cores en FP16. TensorRT 11 a supprimé le drapeau FP16 : la précision est désormais portée par le graphe ONNX lui-même (réseau « strongly typed »). La variante FP16 est donc un ONNX exporté avec des poids FP16, avec des entrées/sorties FP32 pour garder le même code d'exécution.

GPU n°2 — TorchScript et torch.compile. Sur GPU, le gel TorchScript fusionne les convolutions en `cudnn_convolution_relu` (×9) et `cudnn_convolution_add_relu` (×8). torch.compile génère des kernels Triton pour les opérations élémentaires et appelle cuDNN pour les convolutions.

3. Protocole expérimental

Pour une comparaison équitable, tout le protocole est défini une seule fois dans `common.py` et importé par les quatre notebooks :

Élément	Choix
Modèle	torchvision ResNet-18, poids IMAGENET1K_V1, mode eval
Données	50 premières images du jeu de test CIFAR-10, versionnées dans le dépôt (mêmes pixels partout) ; répartition : airplane 5, automobile 3, bird 2, cat 3, deer 5, dog 7, frog 9, horse 4, ship 4, truck 8
Prétraitement	32×32 → 224×224 (bilinéaire), normalisation ImageNet mean (0,485 ; 0,456 ; 0,406), std (0,229 ; 0,224 ; 0,225). Pas de recadrage central, qui couperait l'objet sur une image 32×32

Élément	Choix
10 classes	Les 1000 classes ImageNet sont regroupées en 10 (ex. 118 races de chiens → dog, 59 espèces d'oiseaux → bird). La classe prédite est celle dont le meilleur logit est le plus élevé. « deer » n'existe pas dans ImageNet : les antilopes (ibex, bubale, impala, gazelle) servent d'approximation
Batch	10 images (5 batchs), batch fixe dans les exports ONNX
Répétitions	1 premier appel chronométré à part (compilation), 3 passes de chauffe, puis 10 exécutions mesurées des 50 images
Chronométrage	time.perf_counter ; données déjà sur le périphérique (pas de transfert hôte → GPU) ; torch.cuda.synchronize() après chaque batch sur GPU
Statistiques	moyenne, écart-type (n−1), minimum, maximum, coefficient de variation (CV = σ / moyenne)
Énergie	compteur matériel cumulé NVML (GPU), intégré sur ≥ 3 s d'inférence en boucle après chaque exécution. RAPL (CPU) n'est pas exposé sur les VM Colab
Référence	PyTorch eager mesuré dans chaque notebook, sur la même machine et dans la même session

Environnement Colab mesuré :

	Notebooks CPU (01, 02)	Notebooks GPU (03, 04)
Processeur	Intel(R) Xeon(R) CPU @ 2.20GHz, 2 vCPU, 1 thread PyTorch	Intel(R) Xeon(R) CPU @ 2.00GHz, 2 vCPU
Accélérateur	—	NVIDIA Tesla T4 (16 Go, 70 W), CUDA 13.0, cuDNN 92700
Logiciels	Python 3.13.18, PyTorch 2.11.0+cpu, OpenVINO 2026.4.1	Python 3.13.16, PyTorch 2.11.0+cu130, TensorRT 11.4.0.106

Chaque notebook tourne dans sa propre session Colab, donc potentiellement sur une VM différente. Les deux références eager CPU diffèrent de 13 % (13.1 contre 11.6 img/s) pour un code strictement identique. C'est pourquoi les gains sont toujours calculés par rapport à la référence de la même session.

4. Résultats

4.1 Débit et temps d'inférence

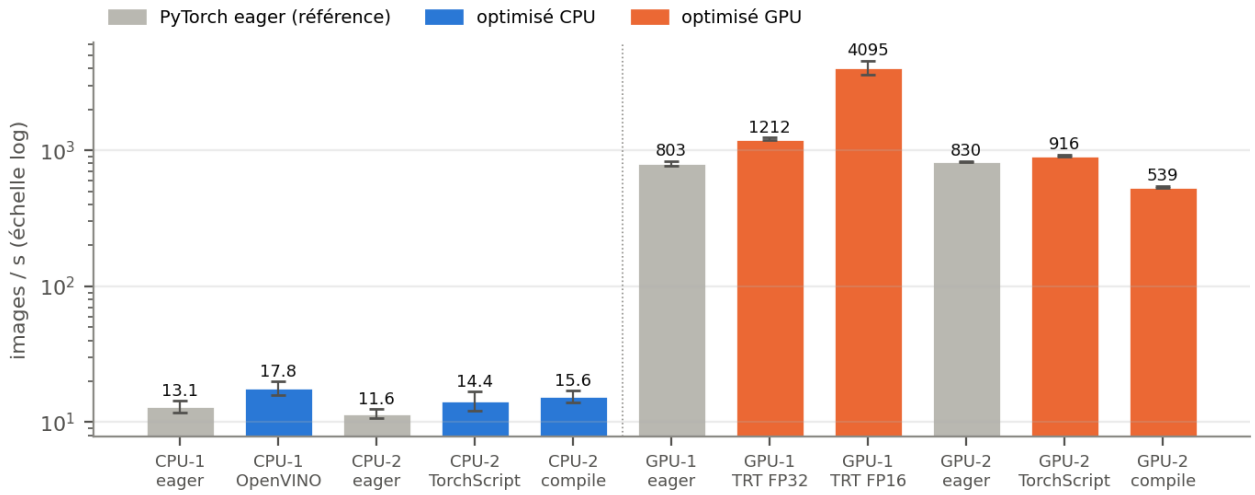


Figure 1 — Débit (images/s) des 11 variantes, moyenne ± écart-type sur 10 exécutions, échelle logarithmique. Gris : référence eager de chaque notebook.

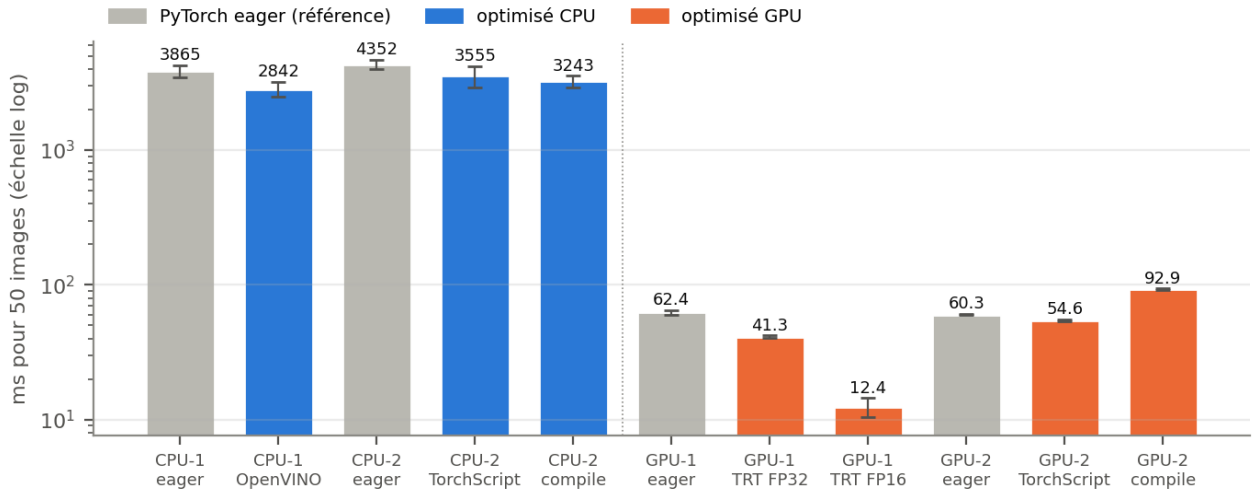


Figure 2 — Temps d'inférence des 50 images (ms), moyenne ± écart-type, échelle logarithmique.

Variante	Débit (img/s) moy. ± σ	min – max	CV	Temps 50 img (ms) moy. ± σ	min – max	Latence batch p95 (ms)
CPU-1 · eager	13.1 ± 1.3	10.9 – 14.3	9.9 %	3864.9 ± 405.1	3495.8 – 4569.3	1109.9
CPU-1 · OpenVINO	17.8 ± 2.0	14.1 – 19.4	11.5 %	2842.0 ± 362.2	2578.2 – 3554.2	805.6
CPU-2 · eager	11.6 ± 0.9	10.3 – 12.6	7.8 %	4351.7 ± 341.0	3963.7 – 4858.6	1229.3
CPU-2 · TorchScript	14.4 ± 2.3	11.1 – 16.0	15.9 %	3555.0 ± 649.1	3127.9 – 4519.7	1023.2
CPU-2 · torch.compile	15.6 ± 1.5	13.5 – 16.8	9.7 %	3242.9 ± 327.0	2973.0 – 3715.7	934.3
GPU-1 · eager	802.8 ± 33.3	712.4 – 826.7	4.1 %	62.4 ± 2.8	60.5 – 70.2	14.0
GPU-1 · TRT FP32	1212.3 ± 30.5	1163.2 – 1256.5	2.5 %	41.3 ± 1.0	39.8 – 43.0	8.7
GPU-1 · TRT FP16	4095.4 ± 480.5	2755.4 – 4340.7	11.7 %	12.4 ± 2.0	11.5 – 18.1	3.6
GPU-2 · eager	829.6 ± 7.1	820.1 – 844.4	0.9 %	60.3 ± 0.5	59.2 – 61.0	12.3
GPU-2 · TorchScript	915.9 ± 13.6	893.6 – 938.7	1.5 %	54.6 ± 0.8	53.3 – 56.0	11.4
GPU-2 · torch.compile	538.6 ± 8.4	520.4 – 549.6	1.6 %	92.9 ± 1.5	91.0 – 96.1	19.3

L'écart CPU/GPU domine tout le reste. À code égal (eager), le T4 traite 72 fois plus d'images par seconde que le CPU Colab (830 contre 11.6 img/s) : 50 images prennent environ 60 ms au lieu de 4.4 s. ResNet-18 demande environ 1,8 milliard de multiplications-additions par image, un

calcul très parallèle que les 2560 cœurs CUDA du T4 absorbent. Le CPU, lui, n'expose qu'un seul thread de calcul à PyTorch. Même la meilleure variante CPU (OpenVINO, 17.8 img/s) reste 45 fois plus lente que le GPU sans optimisation.

4.2 Gain apporté par chaque optimisation

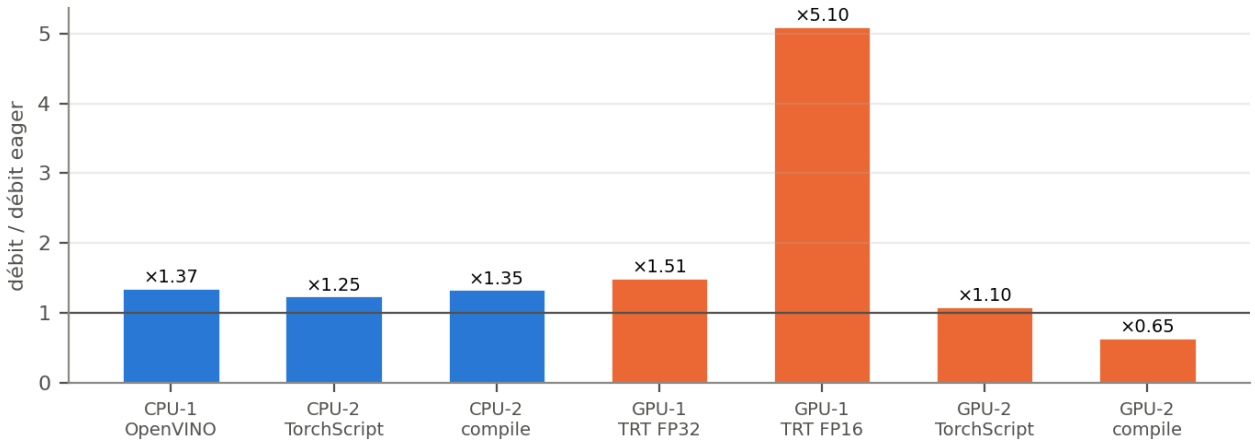


Figure 3 — Accélération par rapport à PyTorch eager mesuré dans la même session (débit / débit eager). Ce rapport neutralise les écarts de matériel entre sessions Colab.

Sur CPU, les trois optimisations apportent un gain modeste et du même ordre : OpenVINO ×1.37, torch.compile ×1.35, TorchScript ×1.25. Les trois appliquent les mêmes transformations de fond : repliement des BatchNorm, fusion convolution + ReLU (+ addition résiduelle) et kernels oneDNN choisis pour le jeu d'instructions (AVX2 ici). Avec un seul cœur, le temps est dominé par les convolutions elles-mêmes, que ni la fusion ni la génération de code n'accélèrent beaucoup. Les intervalles min–max des trois variantes se recouvrent largement : vu la variabilité (§ 4.3), on ne peut pas les départager de façon fiable. Elles sont en revanche toutes nettement au-dessus de la référence eager.

Sur GPU, les écarts sont nets et très reproductibles. TensorRT FP32 gagne ×1.51 : la fusion réduit le nombre de kernels lancés et d'allers-retours en mémoire. Le temps par batch est court (≈ 12 ms en eager), donc ces surcoûts pèsent lourd. TensorRT FP16 atteint ×5.10 (4095 img/s, 12.4 ms pour 50 images). En FP32 le T4 n'utilise que ses cœurs CUDA (≈ 8 TFLOPS) ; le FP16 active les Tensor Cores (≈ 65 TFLOPS) et divise par deux le trafic mémoire. TorchScript gagne ×1.10 grâce aux convolutions cuDNN fusionnées. **torch.compile est en revanche plus lent qu'eager** (×0.65, 18.6 ms par batch contre 12.1). Notre hypothèse, non vérifiée par un profilage : avec `freezing=True`, Inductor convertit les convolutions au format mémoire *channels-last*, et les noyaux cuDNN FP32 du T4 (architecture Turing, sans TF32) y sont moins efficaces. Les conversions de format ajoutent aussi des kernels. Le mode par défaut n'active ni CUDA Graphs ni l'auto-réglage (`max-autotune`, d'ailleurs bridé sur le T4 : « Not enough SMs »). Ce résultat montre qu'une compilation n'est pas un gain garanti : il faut toujours la mesurer face à la référence.

4.3 Stabilité et reproductibilité des mesures

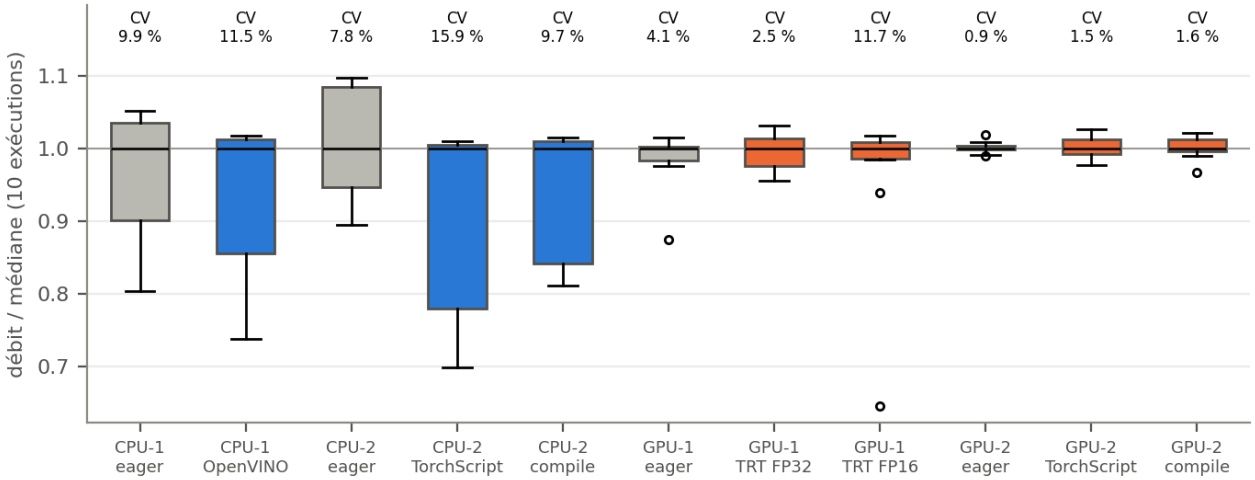


Figure 4 — Distribution du débit sur les 10 exécutions, normalisé par la médiane de chaque variante, et coefficient de variation (CV).

Les mesures GPU sont stables : CV de 0.9 à 4.1 % (hors FP16), et des écarts entre variantes bien supérieurs à leur dispersion, donc des conclusions reproductibles. **Les mesures CPU le sont beaucoup moins** : CV de 7.8 à 15.9 %, avec des exécutions isolées jusqu'à 30 % plus lentes que la médiane. La VM Colab partage en général son cœur physique (2 vCPU = 1 cœur hyperthreadé) avec d'autres processus : la moindre activité concurrente (Jupyter, le système, d'autres locataires de l'hôte) se répercute directement sur le temps. L'écart d'une session à l'autre (§ 3, 13 % entre les deux références eager) est d'ailleurs du même ordre que les gains mesurés.

TensorRT FP16 a un CV de 11.7 % alors que les 9 autres exécutions restent entre 0.94 et 1.02 × la médiane : c'est la **première** exécution mesurée qui est lente (2755 img/s, soit 0.65 × la médiane). La référence eager du même notebook présente le même défaut (0.88 × la médiane à la première exécution). C'est la trace d'une chauffe insuffisante : 3 passes de 50 images ne durent que quelques dizaines de millisecondes sur GPU, trop peu pour que le T4 quitte son état de repos (P8, 9 W au lancement) et atteigne sa fréquence maximale. Avec une passe de 12 ms, ce démarrage pèse lourd en relatif. Une chauffe calibrée en durée (≈ 1 s) plutôt qu'en nombre de passes corrigerait ce biais. En attendant, la médiane est plus représentative que la moyenne pour cette variante.

Conséquence méthodologique : trois exécutions (le minimum demandé) n'auraient pas suffi sur CPU. Avec un CV de 10 %, l'erreur-type de la moyenne de 3 mesures vaut ≈ 6 %, du même ordre que l'écart entre OpenVINO, TorchScript et torch.compile. Nos 10 exécutions la ramènent à ≈ 3 %, et la référence eager mesurée dans la même session supprime l'effet machine.

4.4 Qualité des prédictions

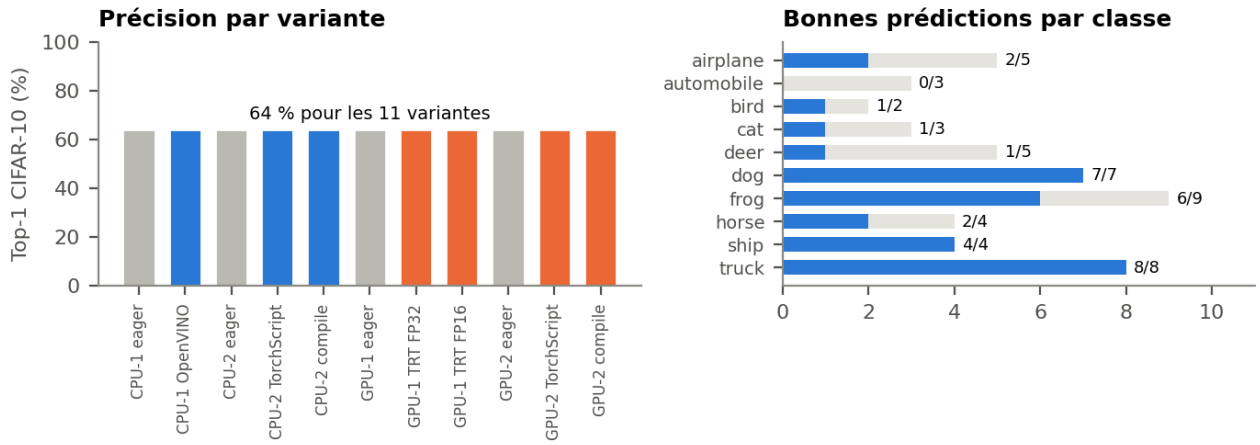


Figure 5 — Précision Top-1 CIFAR-10 de chaque variante (gauche) et détail par classe pour la référence (droite ; la barre grise est le nombre d'images de la classe).

Les 11 variantes obtiennent exactement la même précision, 64 % (32/50). Ce niveau s'explique par le modèle et non par les optimisations : le réseau ImageNet n'a jamais vu d'images CIFAR, qui sont des images 32×32 agrandies 7 fois, donc floues. Souvent, sa classe ImageNet préférée n'appartient à aucun groupe (« sweatshirt », « safety pin », « marmoset »). L'analyse des 18 erreurs montre deux biais du regroupement de classes. 11 erreurs sont prédites *dog*. Ce groupe compte 118 classes ImageNet, et le maximum de 118 logits est mécaniquement plus élevé que celui des groupes de 2 à 5 classes (*horse*, *cat*, *deer*). Ensuite, les 3 voitures sont reconnues comme « moving van », classé *truck*. Les classes bien couvertes sont parfaitement reconnues : *dog* 7/7, *truck* 8/8, *ship* 4/4. Un réentraînement de la dernière couche sur CIFAR-10 relèverait fortement ce score, mais sortirait du cadre « modèle pré-entraîné » commun aux groupes.

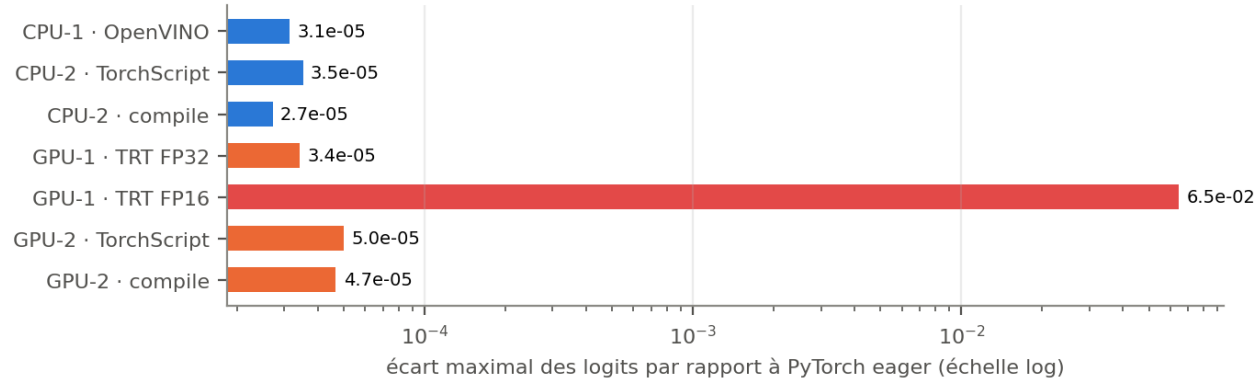


Figure 6 — Écart maximal des 1000 logits par rapport à PyTorch eager (même session), sur les 50 images. Rouge : FP16.

La comparaison des prédictions confirme qu'**aucune optimisation ne dégrade les résultats**. La classe ImageNet prédite est identique pour les 50 images dans les 11 variantes (accord 100 % deux à deux, matrice dans le notebook 05), et donc aussi la classe CIFAR. Toutes les variantes FP32 restent à moins de 5e-05 des logits eager : c'est la différence attendue quand on change l'ordre des sommes flottantes (fusion, Winograd, autres kernels). TensorRT FP16 s'écarte de 0.06, environ 2000 fois plus, du fait de la mantisse 10 bits du demi-flottant. Cet écart reste très inférieur aux marges entre la première et la deuxième classe, d'où aucun changement de prédiction. Sur un jeu plus grand, le FP16 pourrait toutefois inverser quelques cas limites : à vérifier avant un déploiement.

4.5 Consommation énergétique

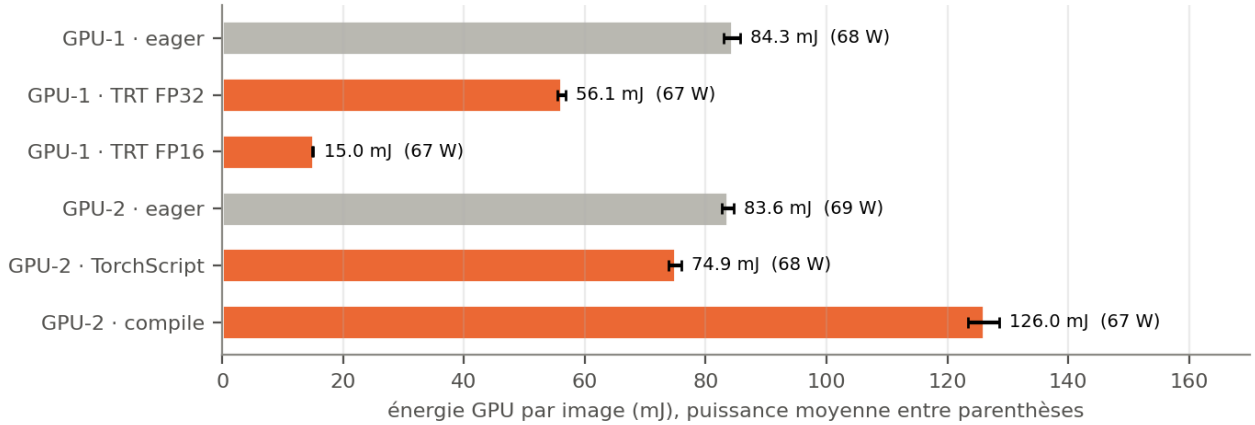


Figure 7 — Énergie consommée par le GPU par image (mJ, moyenne ± écart-type) et puissance moyenne, mesurées avec le compteur NVML.

Pendant l'inférence en boucle, le T4 tourne à pleine charge dans toutes les variantes : puissance moyenne de 67 à 69 W pour une limite de 70 W. La puissance étant quasi constante, l'énergie par image est simplement inversement proportionnelle au débit. TensorRT FP32 consomme 56 mJ par image contre 84 en eager (−33 %), et TensorRT FP16 15 mJ (−82 %). À l'inverse, torch.compile en consomme 51 % de plus. Optimiser le temps d'inférence revient donc ici directement à économiser l'énergie.

Limites de la mesure : NVML ne compte que la carte graphique (ni le CPU hôte, ni la mémoire, ni le refroidissement). Sur CPU, les VM Colab n'exposent pas les compteurs RAPL (`/sys/class/powercap` absent), si bien qu'aucune mesure n'est disponible. Une estimation à partir du TDP n'aurait mesuré que des hypothèses ; nous ne l'avons pas faite.

4.6 Coût de préparation

Variante	Préparation (s)	Nature
CPU · OpenVINO	0.55	lecture ONNX + compilation
CPU · TorchScript	4.65	trace + freeze + optimize_for_inference
CPU · torch.compile	24.5	premier appel (capture + génération C++)
GPU · TensorRT FP32	6.4	construction du moteur (sélection de tactiques)
GPU · TensorRT FP16	19.2	idem, davantage de tactiques candidates
GPU · TorchScript	0.87	trace + freeze
GPU · torch.compile	20.6	premier appel (capture + kernels Triton)

Les moteurs TensorRT et les kernels torch.compile coûtent 6 à 25 s de préparation. C'est négligeable pour un service qui tourne en continu, où le moteur TensorRT est sérialisé une fois puis rechargé. C'est en revanche prohibitif pour un traitement ponctuel de 50 images : sur CPU, la compilation de torch.compile dure autant que 8 traitements complets des 50 images.

5. Conclusion

- **Le choix du matériel prime** : à code égal, le GPU T4 est ≈ 72 fois plus rapide que le CPU Colab, bien plus que ce qu'apporte n'importe quelle optimisation logicielle sur CPU.
- **TensorRT est la chaîne la plus performante** : ×1.5 en FP32 sans aucun écart de prédiction, et ×5.1 en FP16 grâce aux Tensor Cores, pour 82 % d'énergie en moins par image.
- **Sur CPU, OpenVINO, TorchScript et torch.compile gagnent tous ×1.25 à ×1.37**, des écarts entre eux noyés dans la variabilité d'une VM partagée. OpenVINO a l'avantage d'une compilation quasi instantanée.
- **Une compilation n'est pas un gain garanti** : torch.compile est plus lent qu'eager sur le T4 en FP32. Il faut toujours mesurer chaque variante face à une référence dans les mêmes

conditions.

- **Aucune optimisation n'a modifié une prédiction** (précision 64 % partout, accord 100 %). L'écart numérique reste de l'ordre de 10^{-5} en FP32 et 10^{-2} en FP16.
- **Méthode** : les mesures CPU sur Colab ont un CV de 8 à 16 % d'une exécution à l'autre et varient de 13 % d'une session à l'autre. Sur GPU, la première exécution révèle une chauffe trop courte. Répéter les mesures (10 ici), donner la dispersion et normaliser par une référence de même session sont indispensables pour conclure.

Pistes : quantification INT8 (OpenVINO NNCF, TensorRT), torch.compile en FP16 ou avec `mode="max-autotune"` sur un GPU plus récent, variation de la taille de batch (débit contre latence), et transfert hôte → GPU inclus dans la mesure pour un scénario de bout en bout.